FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 4 FEBRUARY 2020 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

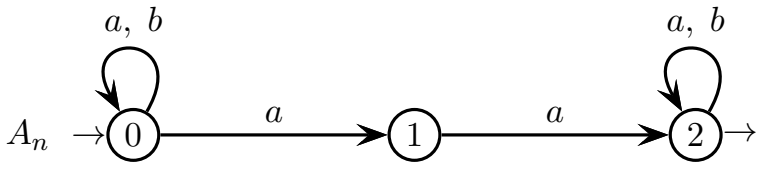
(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A_n below, with a two-letter input alphabet $\Sigma = \{ a, b \}$:



Answer the following questions:

(a) By using the the Berry-Sethi (BS) method, obtain a deterministic automaton A_d equivalent to automaton A_n.

(b) Transform the deterministic automaton A_d into a right-unilinear grammar G.

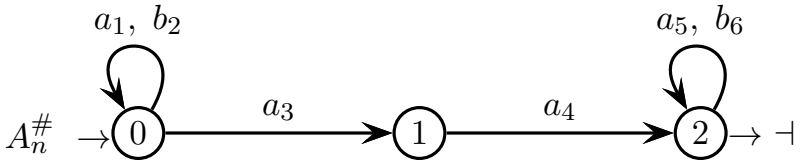
(c) Transform the above grammar G into a system of language equations and solve the system, so as to obtain a regular expression R_1 for the language accepted by automaton A_d.

(d) By using the Brzozowski-McClyskey (BMC) method (node elimination method), obtain a regular expression R_2 from automaton A_d.

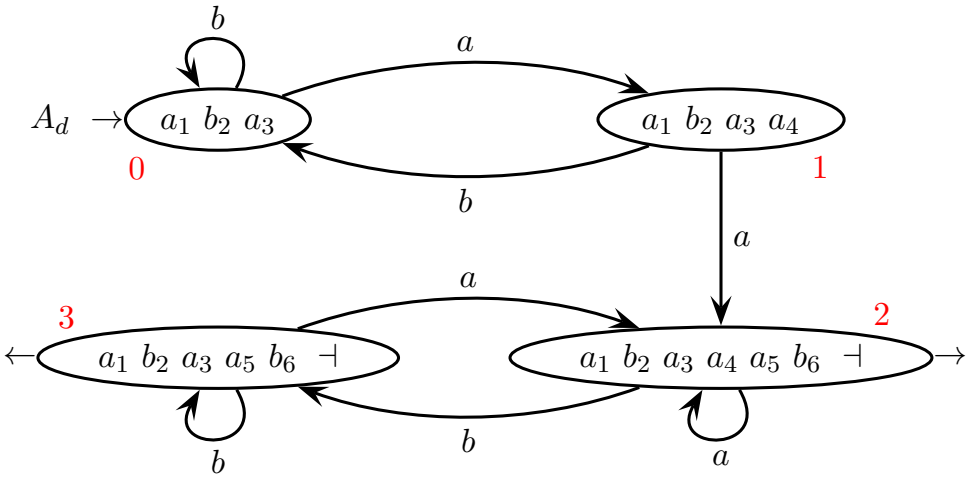
(e) (optional) Determine whether the regular expressions R_1 and R_2 obtained at the previous points (c) and (d) are ambiguous. Justify your answer.

Solution

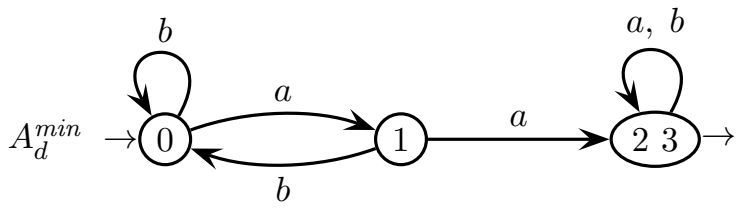
(a) Here is the construction of automaton A_d by means of the Berry-Sethi method:



initials	$a_1\ b_2\ a_3\ \dashv$
generators	followers
a_1	$a_1\ b_2\ a_3$
b_2	$a_1\ b_2\ a_3$
a_3	a_4
a_4	$a_5\ b_6\ \dashv$
a_5	$a_5\ b_6\ \dashv$
b_6	$a_5\ b_6\ \dashv$



Automaton A_d is clean by construction. At a first glance, the two final states 2 and 3 are undistinguishable. Since in the following it is required to manipulate automaton A_d, it is convenient to minimize and rename (as above) the states:



The two non-final states 0 and 1 are distinguishable, because of their a-arcs directed to a non-final and a final state. Thus, we have already reached minimality.

(b) Here is the right-unilinear grammar G (axiom 0), obtained from the minimal version of automaton A_d, i.e., automaton A_d^{min}. The nonterminal alphabet of G coincides with the state set of A_d^{min}, but for compactness the nonterminal 2 3 is shortened as 2:

$$G \begin{cases} 0 \rightarrow b\,0 \mid a\,1 \\ 1 \rightarrow b\,0 \mid a\,2 \\ 2 \rightarrow a\,2 \mid b\,2 \mid \varepsilon \end{cases}$$

For completeness, we can see that the grammar version G' obtained from the non-minimal automaton A_d simply has a difference in the rule for nonterminal 2, and one more rule for nonterminal 3:

$$G' \begin{cases} \dots \\ 2 \rightarrow a\,2 \mid b\,3 \mid \varepsilon \\ 3 \rightarrow a\,2 \mid b\,3 \mid \varepsilon \end{cases}$$

Since the two rules have an identical right part, nonterminals 2 and 3 are equivalent, i.e., it holds $L(2) = L(3)$. In the grammar G', we could drop the rule of 3 and replace nonterminal 3 with 2 anywhere else. Such an operation reduces grammar G' to G and actually has the same effect as minimizing the automaton.

(c) Here is the system of language equations obtained from grammar G:

$$\begin{cases} L_0 = b\,L_0 \cup a\,L_1 \\ L_1 = b\,L_0 \cup a\,L_2 \\ L_2 = a\,L_2 \cup b\,L_2 \cup \varepsilon \end{cases}$$

Here is the solution of the equation system (notice that $(b \cup a) = \Sigma$):

$$\begin{aligned} 1\colon L_0 &= b\,L_0 \cup a\,L_1 \\ 2\colon L_1 &= b\,L_0 \cup a\,L_2 \\ 3\colon L_2 &= (a \cup b)\,L_2 \cup \varepsilon = \Sigma\,L_2 \cup \varepsilon \end{aligned}$$

From eq. (3), through the Arden identity we obtain $L_2 = \Sigma^*$. Then insert this language expression in the place of L_2 into eq. (2), and obtain:

$$L_1 = b\,L_0 \cup a\,\Sigma^*$$

Then replace L_1 with the latter expression into eq. (1), and obtain:

$$L_0 = b\,L_0 \cup a\,\big(b\,L_0 \cup a\,\Sigma^*\big) = (b \cup a\,b)\,L_0 \cup a\,a\,\Sigma^*$$

from which, again through the Arden identity, finally obtain:

$$L_0 = (b \cup a\,b)^*\,a\,a\,\Sigma^*$$

Therefore we have the solution (square brackets indicate optionality):

$$R_1 = (b \mid a\,b)^*\,a\,a\,\Sigma^* = \big([a]\,b\big)^*\,a\,a\,\Sigma^*$$

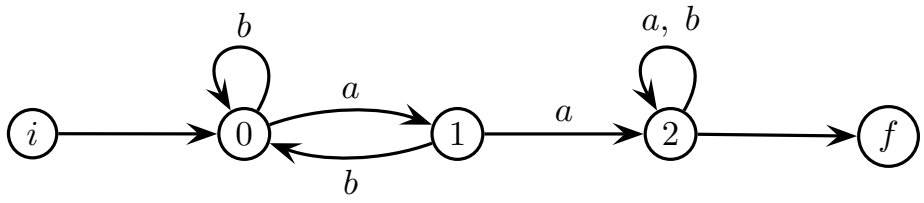
The above solution, obtained through elimination and Arden identity, can be variously transformed by algebraic manipulation, for instance into $b^*\,\big(a\,b^+\big)^*\,a\,a\,\Sigma^*$ or $b^*\,a\,\big(b^+\,a\big)^*\,a\,\Sigma^*$, and possibly into other equivalent forms.

For completeness, we can see that solving the equation system obtained from the non-minimal grammar G' (this system has four equations instead of three) would just require one more substitution step, by first applying the Arden identity to eq. (4), namely:

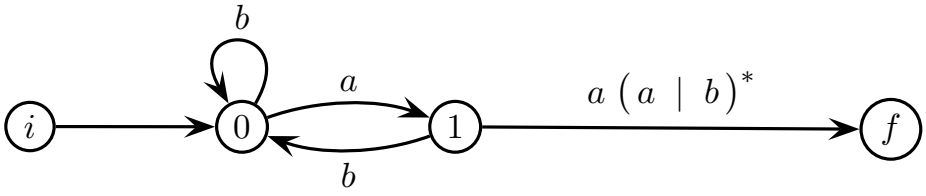
$$\begin{aligned} 4\colon L_3 &= a\,L_2 \cup b\,L_3 \cup \varepsilon = b\,L_3 \cup \big(a\,L_2 \cup \varepsilon\big) \\ &= b^*\,\big(a\,L_2 \cup \varepsilon\big) = b^*\,a\,L_2 \cup b^* \end{aligned}$$

then replacing into eq. (3) and going on solving as before (we omit the details).

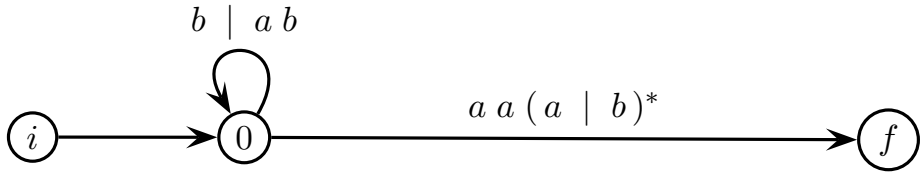
(d) Here is the node elimination, from automaton A_d^{min} (minimal). Normalize:



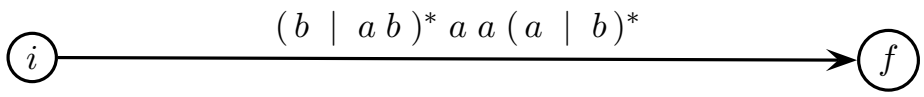
Eliminate node 2:



Eliminate node 1:



Eliminate node 0:

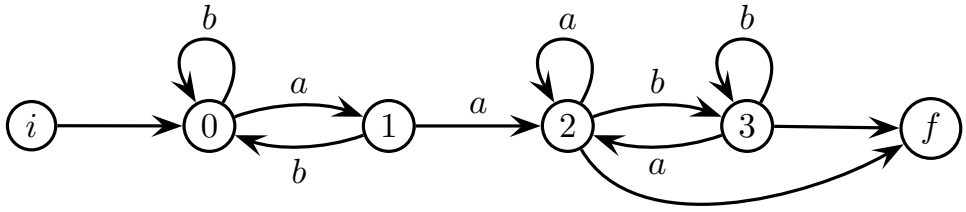


Therefore we have (again notice that $(a \cup b) = \Sigma$):

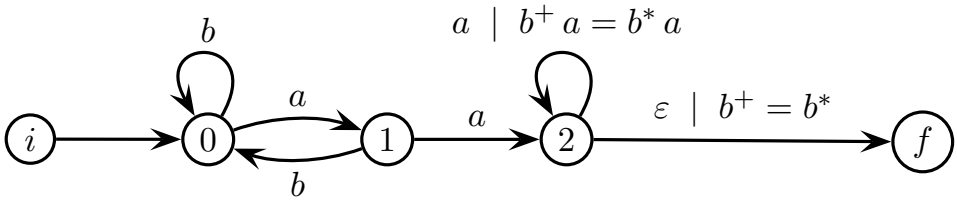
$$R_2 = (b \mid a\,b)^*\,a\,a\,(a \mid b)^* = (b \mid a\,b)^*\,a\,a\,\Sigma^*$$

By chance, it happens that expressions R_2 and R_1 are structurally identical, i.e., $R_2 = R_1$, which implies $L(R_2) = L(R_1)$ as expected. Of course, eliminating the nodes in a different order might yield a structurally different version of expression R_2, though still equivalent to R_1 (see also point (c)).

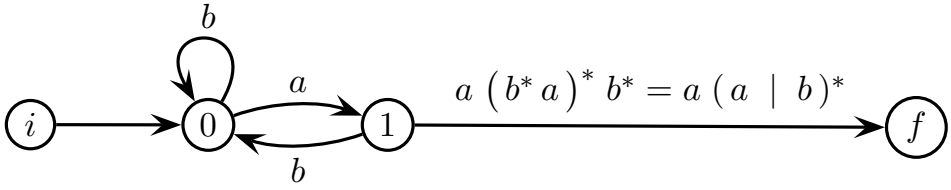
For completeness, we could see that eliminating the nodes from the non-minimal automaton A_d would also yield an equivalent result. Here is the starting machine, already normalized:



Eliminate node 3:



Eliminate node 2:



Now the rest is the same as before, and the result identical (we omit the details).

(e) The regular expression $R_1 = (\, b \mid a\ b\,)^*\ a\ a\ (\, a \mid b\,)^*$ is not ambiguous. To see this, we show that the symbols of a string in the language $L\,(R_1)$ match the symbols in the numbered version of R_1 in a unique way. The numbered version of R_1, i.e., $R_1^{\#}$, is as follows (expanding the alphabet):

$$R_1^{\#} = (\, b_1 \mid a_2\ b_3\,)^*\ a_4\ a_5\ (\, a_6 \mid b_7\,)^*$$

The strings of language $L\,(R_1)$ include at least one digram $a\ a$. The first (left-most) occurrence of such a digram has to match the numbered symbols $a_4\ a_5$, thus any subsequent symbol a or b necessarily matches a numbered symbol a_6 or b_7 in a unique way. A similar reasoning can be inductively applied to any substring that precedes the first occurrence of the digram $a\ a$, as follows:

- an initial b matches b_1
- an a (initial or not) followed by a b matches $a_2\ b_3$
- two adjacent b match $b_1\ b_1$ or $b_3\ b_1$, but the latter match is distinguished from the former one by the triplet $a_2\ b_3\ b_1$ (as $b_1\ b_1$ may not be preceded by a_2)

Thus each non-numbered symbol matches a numbered one in a unique way. Expression R_2 structurally coincides with R_1, thus it is not ambiguous either.

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language L_{Dyck} with two pairs of matching open / closed parentheses, i.e., a / b and c / d. A well-known non-ambiguous grammar G, over the terminal alphabet $\Sigma = \{\ a,\ b,\ c,\ d\ \}$, that generates language L_{Dyck} is shown below (axiom S):

$$G \begin{cases} S \ \rightarrow\ a\ S\ b\ S \\ S \ \rightarrow\ c\ S\ d\ S \\ S \ \rightarrow\ \varepsilon \end{cases}$$

Answer the following questions:

(a) Write two *BNF* grammars G_1 and G_2 (no matter if ambiguous), respectively for the languages L_1, $L_2 \subset L_{Dyck}$ defined as follows:

(i) L_1 contains all the strings of L_{Dyck} except those that have the digram $a\,c$

(ii) L_2 contains all the strings of L_{Dyck} except those that have the digram $d\,c$

Suitably modify grammar G, but do not use more than three nonterminals for each grammar. Verify that the following is true (draw the trees when possible):

(i) $a\,c\,d\,b \notin L_1$ and $c\,a\,b\,d \in L_1$

(ii) $c\,d\,c\,d \notin L_2$ and $c\,d\,a\,b \in L_2$

(b) Now consider the regular expression R below:

$$R = (\ a\ \mid\ b\ )^*\ (\ c\ \mid\ d\ )^*$$

Write a non-ambiguous *BNF* grammar G_3 that generates the language $L_3 \subset L_{Dyck}$ below:

$$L_3 = L_{Dyck}\ \cap\ L\,(R)$$

Suitably modify grammar G, but do not use more than three nonterminals. Reasonably argue that grammar G_3 is not ambiguous.

(c) (optional) Consider the well-known *EBNF* grammar below (axiom S), which generates the Dyck language with only one parenthesis pair, i.e., a / b:

$$S \rightarrow \left(\ a\ S\ b\ \right)^*$$

Suitably modify this grammar, extend the terminal alphabet to $\Sigma = \{\ a,\ b,\ c,\ d\ \}$, and write two *EBNF* grammars G^E and G_3^E that have the following properties:

(i) grammar G^E is weakly equivalent to grammar G

(ii) grammar G_3^E is weakly equivalent to grammar G_3

For both grammars G^E and G_3^E, do not use more than *one* nonterminal, i.e., S.

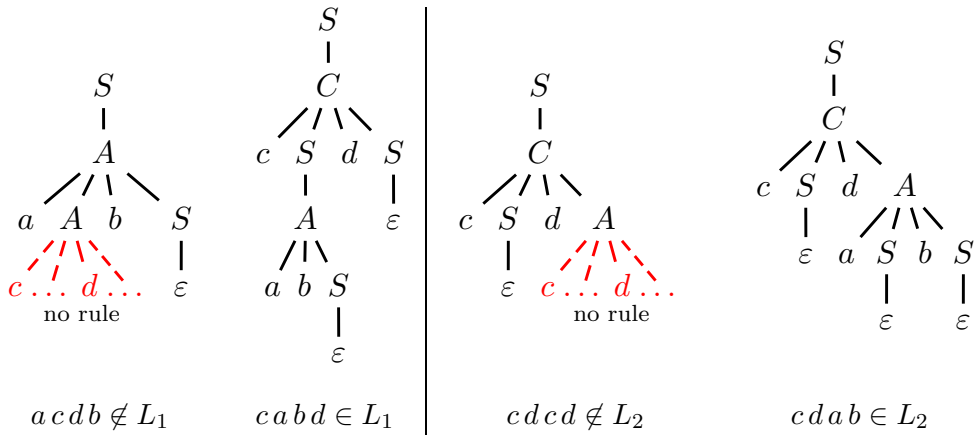
Solution

(a) Here are the two grammars G_1 and G_2 for the Dyck languages without digram $a\,c$ and $d\,c$, respectively. Both grammars are non-ambiguous (axiom S):

$$G_1 \begin{cases} S \to A \mid C \mid \varepsilon \\ A \to a\ A\ b\ S \mid a\ b\ S \\ C \to c\ S\ d\ S \end{cases} \qquad G_2 \begin{cases} S \to A \mid C \mid \varepsilon \\ A \to a\ S\ b\ S \\ C \to c\ S\ d\ A \mid c\ S\ d \end{cases}$$

Both nonterminals A and C generate a (sub)set of Dyck strings, starting from a nest $a\,b$ and a nest $c\,d$, respectively. In the grammar G_1, nonterminal A is unable to immediately insert a nest $c\,d$ inside a nest $a\,b$, thus a Dyck string may not exhibit the digram $a\,c$. In the grammar G_2, nonterminal C is unable to concatenate two nests $c\,d$, thus a Dyck string may not exhibit the digram $d\,c$. Therefore the constraints required for the two languages are fulfilled.

Here are the (tentative) syntax trees of the four sample strings:



The trees of the invalid strings cannot be completed (in red), as expected.

Other versions of grammars G_1 and G_2 may exist, possibly with fewer nonterminals. For instance G'_1 and G'_2, each with two nonterminals only (axiom S):

$$G'_1 \begin{cases} S \to A \mid c\ S\ d\ S \mid \varepsilon \\ A \to a\ A\ b\ S \mid a\ b\ S \end{cases} \qquad G''_1 \begin{cases} S \to A \mid c\ S\ d\ S \mid \varepsilon \\ A \to a\ A\ b\ S \mid \varepsilon \end{cases}$$

$$G'_2 \begin{cases} S \to A \mid c\ S\ d\ A \mid c\ S\ d \mid \varepsilon \\ A \to a\ S\ b\ S \end{cases} \qquad G''_2 \begin{cases} S \to A \mid c\ S\ d\ A \mid \varepsilon \\ A \to a\ S\ b\ S \mid \varepsilon \end{cases}$$

Both new grammars G'_1 and G'_2 (still non-ambiguous) are obtained from the previous ones by removing the copy rule $S \to C$, through a standard grammar transformation. In general the number of nonterminals would stay the same, yet here by chance the transformation causes the nonterminal C to become useless, thus to be removable, and so reduces the nonterminal alphabet size. The two corresponding versions G''_1 and G''_2 shown aside are ambiguous, since each of

them can derive the empty string ε as $S \Rightarrow \varepsilon$ and $S \Rightarrow A \Rightarrow \varepsilon$, i.e., through two different derivations, but in return they are even more compact.

(b) Intersecting the Dyck language with the regular language $L(R)$ implies that in a valid string first (on the left) there are all the pairs $a\,b$, then (on the right) all the pairs $c\,d$. A simple non-ambiguous grammar G_3 is the following (axiom S):

$$G_3 \begin{cases} S \;\to\; A\;C \;\mid\; A \;\mid\; C \;\mid\; \varepsilon \\ A \;\to\; a\;A\;b\;A \;\mid\; a\;A\;b \;\mid\; a\;b\;A \;\mid\; a\;b \\ C \;\to\; c\;C\;d\;C \;\mid\; c\;C\;d \;\mid\; c\;d\;C \;\mid\; c\;d \end{cases}$$

Languages $L(A)$ and $L(C)$ are Dyck with only pairs $a\,b$ and $c\,d$, respectively. The axiomatic rules can concatenate them, as requested, or generate them separately (if either one is missing in the concatenation), plus the empty string ε.

Since these two Dyck languages have disjoint terminal alphabets, in the axiomatic rules there is no possibility of union or concatenation ambiguity, except perhaps for the empty string ε. However, even string ε is unambiguous, as both nonterminals A and C are non-nullable and string ε is generated only once by the axiom itself. Therefore grammar G_3 is reasonably not ambiguous.

An even simpler grammar version G'_3 (still non-ambiguous) is as follows:

$$G'_3\colon S \to A\;C \qquad A \to a\;A\;b\;A \;\mid\; \varepsilon \qquad C \to c\;C\;d\;C \;\mid\; \varepsilon$$

Differently from grammar G_3, grammar G'_3 does not have any copy rules, but both nonterminals A and C of G'_3 are nullable. Since the alphabets of languages $L(A)$ and $L(C)$ are disjoint, there cannot be concatenation ambiguity.

(c) Here is a possible *EBNF* grammar G^E (non-ambiguous, with one nonterminal):

$$G^E\colon S \to \big(\,a\;S\;b \;\mid\; c\;S\;d\,\big)^*$$

We know that an *EBNF* rule with a star in the right side can be expanded into arbitrarily many *BNF* rules, by deriving the star. Thus grammar G^E has the rules $S \to \varepsilon$, $S \to a\;S\;b$, $S \to c\;S\;d$, $S \to a\;S\;b\;a\;S\;b$, $S \to a\;S\;b\;c\;S\;d$, $S \to c\;S\;d\;a\;S\;b$, $S \to c\;S\;d\;c\;S\;d$, and so on with a right side of increasing length, which concatenate parenthesis nests of type $a\,S\,b$ and $c\,S\,d$ in any number and order. The recursive nonterminal S inserts such nests at an arbitrary depth. Therefore grammar G^E generates all the Dyck strings, and only those.

Instead, there does not exist any grammar G_3^E with only one nonterminal. In fact, suppose such a grammar G_3^E exists. Then the unique nonterminal of G_3^E has to be recursive, as it should generate parenthesis nests – of both types $a\,b$ and $c\,d$ – at an arbitrary depth. Consequently, by recursion it could unavoidably generate also a nest $a\,c\,d\,b$ of depth two, though by definition such a nest cannot belong to language $L(G_3)$. Thus a one-nonterminal grammar G_3^E does not exist. It is easy however, to write a grammar G_3^E with more than one nonterminal:

$$G_3^E\colon S \to A\;C \qquad A \to \big(\,a\;A\;b\,\big)^* \qquad C \to \big(\,c\;C\;d\,\big)^*$$

Grammar G_3^E is quite similar to grammars G_3 and G'_3, although it is more compact than those, since it takes advantage of the *EBNF* form.

2. A programming language fragment includes variables and constants, relational and boolean expressions, assignment and conditional statements, specified as follows:

   - A conditional statement, within the keywords "if" and "fi", orderly contains:
     - a condition, which consists of any expression
     - a clause *then*, with keyword "then" and a non-empty list of statements
     - zero, one or more clauses *elsif*, with keyword "elsif"; each such clause contains a condition and only one clause *then*, which is as specified before
     - an optional clause *else*, with keyword "else" and a non-empty list of statements
   - An expression contains the following operators: equality "=", negation "not", n-ary conjunction "and" and n-ary disjunction "or" (both with $n \geq 2$), and implication "imp" (only binary). As usual, round parentheses may enclose subexpressions and allow the user to override the default operator precedence.
   - The precedence of the operators, in decreasing order from the highest, i.e., maximal priority, to the lowest, i.e., minimal priority, is the following: equality, negation, conjunction, disjunction and implication.
   - An assignment statement, with the operator ":=", assigns an expression (on the right-hand side) to a variable (on the left-hand side).
   - The lexical tokens for the identifiers are schematized by terminal id, which does not need to be further expanded.
   - A phrase of the language is a non-empty list of assignment and conditional statements, each of them terminated by a semicolon.

   Here is a sample phrase of the language, from which you should infer any detail not included in the above specifications of the language syntax:

```
a := b;
if not a = c and b = c imp e and f and g and ( d or h )
   then
      c := e;
      e := f and g;
   elsif not e
      then
         if ( c = a or f ) and g
            then b := a;
            else b := e;
         fi;
         e := d;
fi;
```

   Answer the following questions:

   (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the language sketched above.

   (b) To test the correctness of your grammar, draw the syntax subtree of the expression at line two of the sample phrase, i.e., the subtree of the expression:

```
not a = c and b = c imp e and f and g and ( d or h )
```

Solution

(a) Here is a possible and reasonable formulation of the grammar requested for the sketched language fragment (axiom FRAG):

$$
\left\{
\begin{array}{rcll}
\langle \mathsf{FRAG} \rangle & \rightarrow & \Big(\big(\langle \mathsf{C\_STM} \rangle \mid \langle \mathsf{A\_STM} \rangle \big) \text{ ';'} \Big)^{+} & \\
\hline
\langle \mathsf{C\_STM} \rangle & \rightarrow & \mathsf{if}\ \langle \mathsf{EXPR} \rangle\ \langle \mathsf{CLAUSES} \rangle\ \mathsf{fi} & \\
\langle \mathsf{A\_STM} \rangle & \rightarrow & \mathsf{id}\ \text{':='}\ \langle \mathsf{EXPR} \rangle & \\
\hline
\langle \mathsf{CLAUSES} \rangle & \rightarrow & \langle \mathsf{THEN} \rangle\ \langle \mathsf{ELSIF} \rangle^{*}\ \big[\ \langle \mathsf{ELSE} \rangle\ \big] & \\
\hline
\langle \mathsf{EXPR} \rangle & \rightarrow & \langle \mathsf{IMP} \rangle\ \big[\ \mathsf{imp}\ \langle \mathsf{IMP} \rangle\ \big] & \text{// min priority} \\
\langle \mathsf{IMP} \rangle & \rightarrow & \langle \mathsf{TRM} \rangle\ \big(\ \mathsf{or}\ \langle \mathsf{TRM} \rangle\ \big)^{*} & \\
\langle \mathsf{TRM} \rangle & \rightarrow & \langle \mathsf{FCT} \rangle\ \big(\ \mathsf{and}\ \langle \mathsf{FCT} \rangle\ \big)^{*} & \\
\langle \mathsf{FCT} \rangle & \rightarrow & \big[\ \mathsf{not}\ \big]\ \langle \mathsf{EQU} \rangle & \\
\langle \mathsf{EQU} \rangle & \rightarrow & \langle \mathsf{OBJ} \rangle\ \big[\ \text{'='}\ \langle \mathsf{OBJ} \rangle\ \big] & \text{// max priority} \\
\langle \mathsf{OBJ} \rangle & \rightarrow & \mathsf{id} \mid \text{'('}\ \langle \mathsf{EXPR} \rangle\ \text{')'} & \\
\hline
\langle \mathsf{THEN} \rangle & \rightarrow & \mathsf{then}\ \langle \mathsf{FRAG} \rangle & \\
\langle \mathsf{ELSIF} \rangle & \rightarrow & \mathsf{elsif}\ \langle \mathsf{EXPR} \rangle\ \langle \mathsf{THEN} \rangle & \\
\langle \mathsf{ELSE} \rangle & \rightarrow & \mathsf{else}\ \langle \mathsf{FRAG} \rangle &
\end{array}
\right.
$$

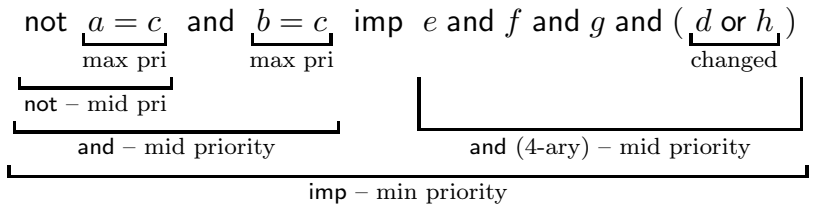
Square brackets indicate optionality. A program fragment is modeled as a non-empty list of assignment and conditional statements, each of which is terminated by a semicolon. The two statement types are modeled by separate rules.

The clauses of the conditional statement *if* are ordered and modeled according to the specifications: *then* is mandatory and unique, *elsif* is optional and may be multiple, and *else* is optional and unique. Both clauses *then* and *else* contain a generic (non-empty) statement list, while the clause *elsif* contains a mandatory and unique clause *then*, which of course is selected by an expression.

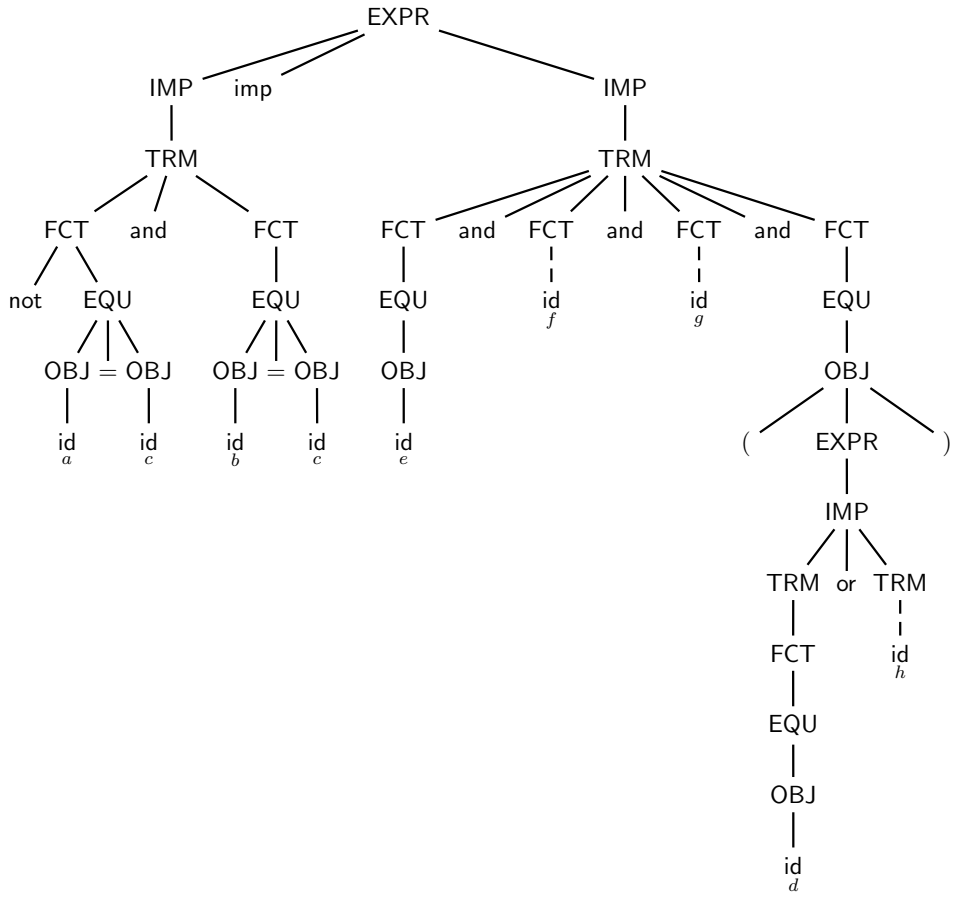
Expressions contain all the required operators. Conjunction and disjunction are modeled as n-ary operators (with $n \geq 2$), implication and equality are modeled as binary, and of course negation is strictly unary. The associativity of conjunction and disjunction, each of which may have more than two operands, is not modeled, as the language specifications do not state anything about such an issue. This choice allows us to write *EBNF* iterative rules (not left or right recursive rules) for both operators "and" and "or", and thus to simplify the grammar. The default operator precedence is also modeled, by layering the corresponding rules.

Most grammar rules are of a standard unambiguous type. Notice that a dangling clause *elsif* or *else* is impossible, since the conditional statement *if* has the closing keyword "fi" and the clause *elsif* may not immediately contain clauses *elsif* or *else*, but only one clause *then*. Thus, the ambiguity form of a dangling clause may not occur. In conclusion, the whole grammar is reasonably unambiguous. It is even likely to be *ELL* (the reader may check), *a-fortiori* unambiguous.

(b) First notice that the operator precedence in the expression is as follows (under-bracketing indicates the operator precedence, whether by default or explicit):



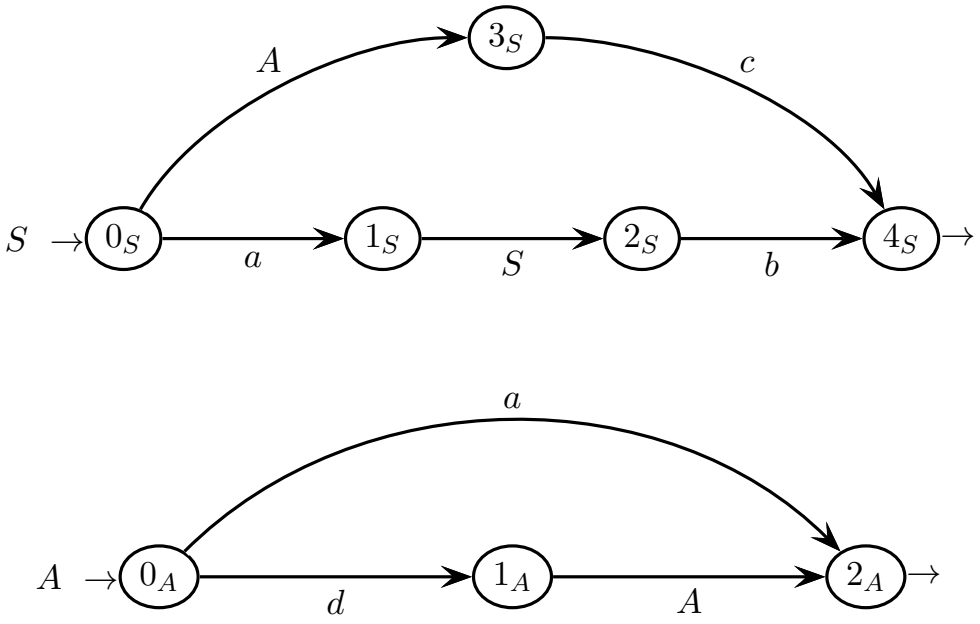
There is one 4-ary operator "and", while all the other operators are binary or unary. By default, the operators "=" have maximum priority, the operators "not" and "and" have intermediate priority ("not" higher than "and"), and the operator "imp" has minimum priority. The expression contains a round bracket pair, which changes the default operator precedence and explicitly gives to the operator "or" a higher priority than to the "and" containing the "or". Here is the syntax tree of the expression, which is rooted at nonterminal EXPR:



To simplify, the tree is partially condensed, by shortening the strictly linear tree branches (dashed) that already appear on the left in their full extension. The tree correctly models the deep structure of the expression with the specified operator precedence. The whole chain of three logical products (and) is appended to one node (TRM), since the associativity of the n-ary operators is unspecified. To help mapping tree and expression, the variables are listed under their identifiers.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, over the four-letter terminal alphabet $\Sigma = \{\ a,\ b,\ c,\ d\ \}$ and the two-letter nonterminal alphabet $V = \{\ A,\ S\ \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

(a) Draw the complete pilot automaton of grammar G. Show that grammar G is of type $ELR\,(1)$. Justify your answer.

(b) Find all the guide sets on the call arcs and exit arrows of the $PCFG$ of grammar G. Show that grammar G is not of type $ELL\,(1)$. Justify your answer.

(c) Simulate the ELR recognition of the sample valid string below:

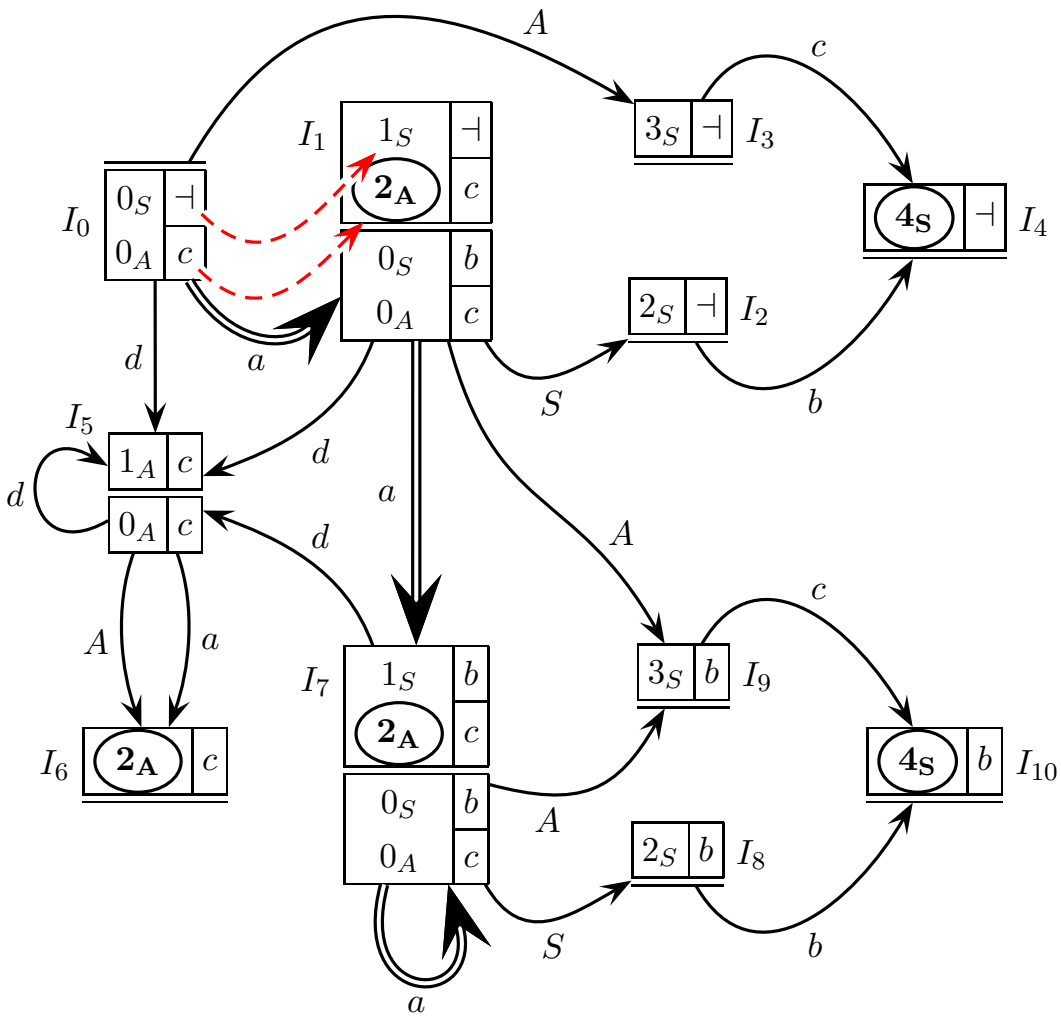
$$a\ a\ c\ b \in L\,(G)$$

and build the corresponding syntax tree. Show the correspondence between each subtree and each reduction move in the simulation.

(d) (optional) Determine whether grammar G is $ELL\,(k)$ for some $k \geq 2$. Justify your answer.

Solution

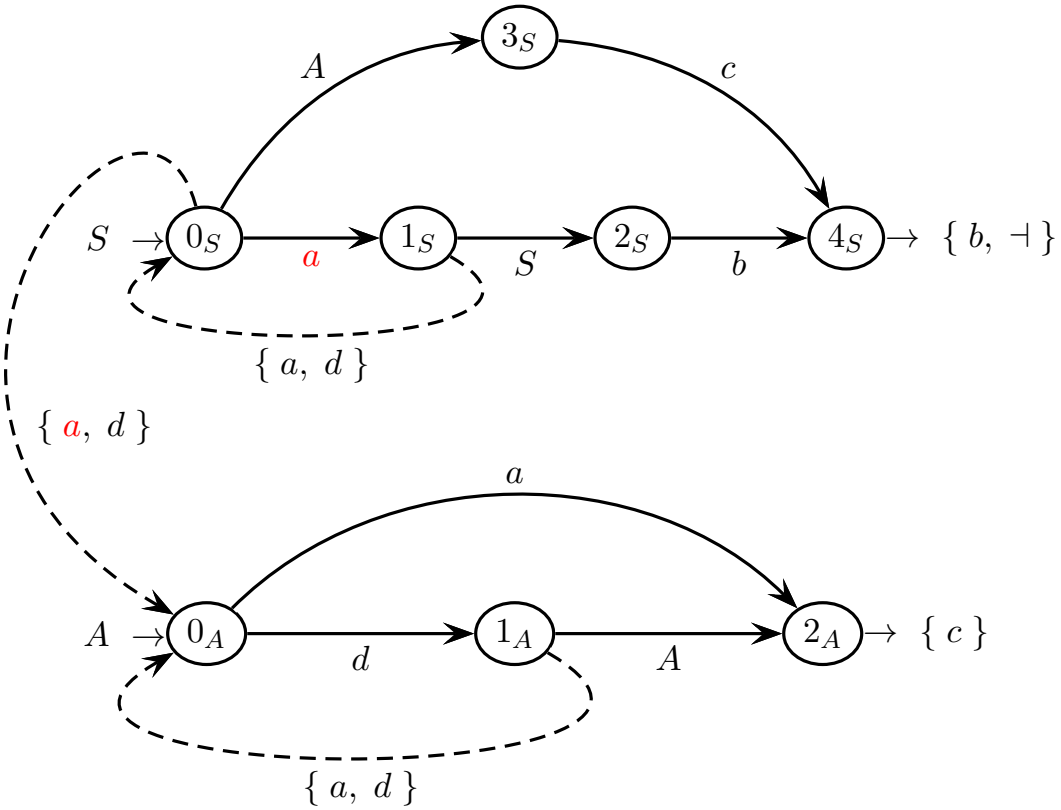
(a) Here is the complete pilot automaton of grammar G, with eleven m-states:



There are no conflicts of any kind, i.e., shift-reduce, reduce-reduce or convergence (as there are no convergent transitions either), thus grammar G is $ELR(1)$.

The pilot does not have the *STP*, since some m-states have a base with two items, i.e., I_1 and I_7. In fact, any transition to one of such m-states is double, i.e., $I_0 \xrightarrow{a} I_1$, $I_1 \xrightarrow{a} I_7$ and $I_7 \xrightarrow{a} I_7$ (double lined). The two sub-transitions of $I_0 \xrightarrow{a} I_1$ are shown, i.e., $\langle 0_S, \dashv \rangle \dashrightarrow \langle 1_S, \dashv \rangle$ and $\langle 0_A, c \rangle \dashrightarrow \langle 2_A, c \rangle$ (dashed red). Those of $I_1 \xrightarrow{a} I_7$ and $I_7 \xrightarrow{a} I_7$ are similar, thus we omit them.

(b) Here is the *PCFG*, with all the guide sets on the call arcs and exit arrows (those on the shift arcs are trivial and not shown):



Axiom S is recursive and may be followed by terminal b, plus the terminator $\dashv$. Nonterminal A is necessarily followed by terminal c. Thus the guide sets on an exit site of S and A are $\{ b, \dashv \}$ and $\{ c \}$, respectively.

Nonterminal A is non-nullable and its initials are terminals a and d. Thus the guide set of any call site to A is $\{ a, d \}$. Also nonterminal S is non-nullable and its initials are those of A plus terminal a, which does not add anything to the initials of A. Thus the guide set of the unique call site to S is still $\{ a, d \}$.

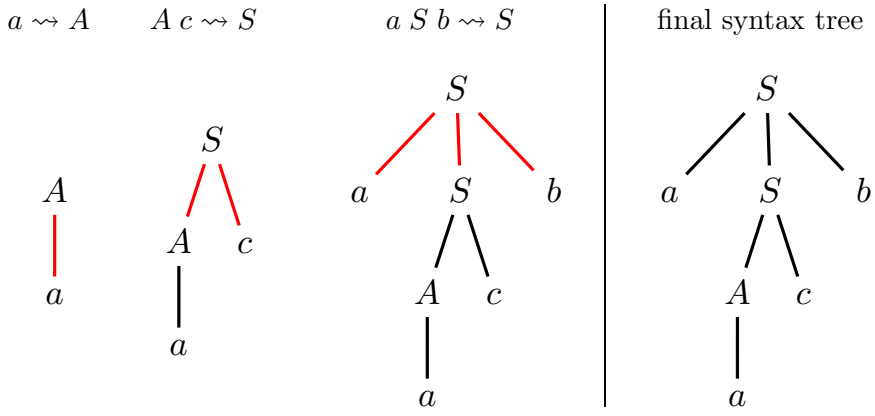
There is one conflict in the state 0_S on terminal a (and there are not any other conflicts). However, such a conflict is sufficient to make grammar G non-$ELL(1)$. This was expected, as the pilot does not have the *STP*, as transitions $I_0 \xrightarrow{a} I_1$, $I_1 \xrightarrow{a} I_7$ and $I_7 \xrightarrow{a} I_7$ (self-loop) are multiple (more precisely double), though not convergent (see also what was already observed at point (a)).

(c) Here is the *ELR* simulation of the sample valid string $a\,a\,c\,b$ (nine moves in total):

simulation table for the *ELR* analysis

stack of alternated macro-states and grammar symbols	*input left to be scanned*	*move executed*
I_0 (empty stack)	$a\ a\ c\ b\ \dashv$	initial configuration
$I_0\ a\ I_1$	$a\ c\ b\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_1$
$I_0\ a\ I_1\ a\ I_7$	$c\ b\ \dashv$	terminal shift: $I_1 \xrightarrow{a} I_7$
$I_0\ a\ I_1$	$c\ b\ \dashv$	reduction: $a \rightsquigarrow A$
$I_0\ a\ I_1\ A\ I_9$	$c\ b\ \dashv$	nonterminal shift: $I_1 \xrightarrow{A} I_9$
$I_0\ a\ I_1\ A\ I_9\ c\ I_{10}$	$b\ \dashv$	terminal shift: $I_9 \xrightarrow{c} I_{10}$
$I_0\ a\ I_1$	$b\ \dashv$	reduction: $A\ c \rightsquigarrow S$
$I_0\ a\ I_1\ S\ I_2$	$b\ \dashv$	nonterminal shift: $I_1 \xrightarrow{S} I_2$
$I_0\ a\ I_1\ S\ I_2\ b\ I_4$	$\dashv$	terminal shift: $I_2 \xrightarrow{b} I_4$
I_0	$\dashv$	reduction: $a\ S\ b \rightsquigarrow S$
I_0 (empty stack)	empty tape	reduction to axiom
acceptance condition is verified – string $a\,a\,c\,b$ is valid		

The acceptance condition is eventually verified. There are three reduction moves, which correspond to three subtrees to be orderly connected bottom-up:



The three elementary subtrees corresponding to each reduction move (shown on top) are orderly listed, connected and colored in red, along with the final tree.

(d) There is only one *PCFG* conflict in the state 0_S on terminal a, between the shift arc $0_S \xrightarrow{\{\, a \,\}} 1_S$ and the call arc $0_S \xrightarrow{\{\, a,\, d \,\}} 0_A$. The conflict vanishes with look-ahead distance $k = 2$, as the guide sets from 0_S become $0_S \xrightarrow{\{\, a\,a,\, a\,d \,\}} 1_S$ and $0_S \xrightarrow{\{\, a\,c,\, d\,a,\, d\,d \,\}} 0_A$, which now are disjoint. Thus grammar G is $ELL\,(2)$.
To study grammar G more in depth, we can represent it by means of *EBNF* rules structurally equivalent to the machine net, as follows (axiom S):

$$G \begin{cases} S \;\rightarrow\; a\; S\; b \;\mid\; A\; c \\ A \;\rightarrow\; d\; A \;\mid\; a \end{cases}$$

The rule of nonterminal A is right-linear and the axiomatic rule is linear self-embedding. Thus the strings of language $L\,(A)$ are of type $d^m\; a$ with $m \geq 0$, i.e., regular of type $d^*\; a$, and those of language $L\,(S)$ are of type $a^n\; L(A)\; c\; b^n$ with $n \geq 0$. Therefore the language of grammar G is as follows:

$$L\,(G) = \{\; a^n\; d^m\; a\; c\; b^n \mid \quad m,\, n \geq 0 \;\}$$

To parse the strings of type $a^n\; d^0\; a\; c\; b^n = a^n\; a\; c\; b^n = a^{n+1}\; c\; b^n$, a recursive descent (predictive) analyzer based on grammar G needs a look-ahead window of size $k = 2$. In fact, the analyzer has to locate the endpoint of the prefix a^n, because it needs to read and push such a prefix, and later match the stacked prefix with the suffix b^n, while it should not push the exceeding letter a. To avoid such an additional push however, the analyzer needs to safely identify the $(n+1)$-th letter a before shifting the input head, that is, it needs to look ahead for such an a (so far $k = 1$). But for the identification to be certain, the analyzer needs to look ahead one letter more for a c, thus a window size $k = 2$ is necessary. For all the other strings, which are of type $a^n\; d^+\; a\; c\; b^n$, a size $k = 1$ is sufficient.

4 Language Translation and Semantic Analysis 20%

1. The grammar G_s below generates the source language L_s of the bit strings of even length ≥ 2, over the binary source alphabet $\Sigma = \{\ 0,\ 1\ \}$ (axiom S):

$$G_s \begin{cases} S \ \rightarrow\ 0\,Q \ \mid\ 1\,R \\ Q \ \rightarrow\ 0\,S \ \mid\ 1\,S \ \mid\ 0 \ \mid\ 1 \\ R \ \rightarrow\ 0\,S \ \mid\ 1\,S \ \mid\ 0 \ \mid\ 1 \end{cases}$$

   We intend to define and compute a translation τ that maps the strings of language L_s onto the strings in base four, hence strings over the four-digit destination alphabet $\Delta = \{\ 0,\ 1,\ 2,\ 3\ \}$, that have the same numerical value.

   For instance the binary string $1\,0\,0\,1\,1\,1\,0\,0$, which has decimal value 156, is mapped by translation τ to the base-four string $2\,1\,3\,0$, which has the same decimal value:

$$\tau\left(1\,0\,0\,1\,1\,1\,0\,0\right) = 2\,1\,3\,0 \qquad \text{– both strings have decimal value 156}$$

   Answer the following questions (use the tables / drawings / spaces on the next pages):

   (a) Write the destination grammar G_d (or the translation grammar G_τ) that defines the translation τ described above.

   (b) Draw the destination (or translation) syntax tree for the sample source string:

$$1\,0\,0\,1\,1\,1\,0\,0$$

   (c) Write a regular translation expression (rational expression) R_τ that defines the above translation τ and design a finite-state transducer A_τ that computes it.

   (d) (optional) Write the recursive descent *ELL*-based procedures, only for the non-terminals S and R, that read the source string, compute translation τ and output the destination string.

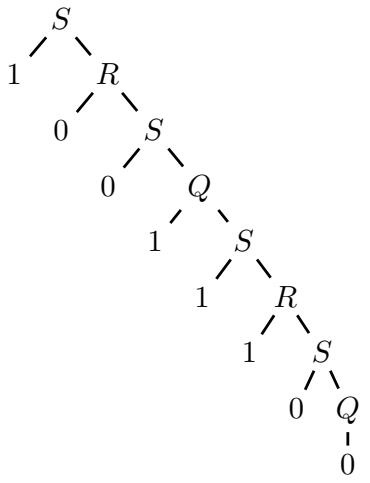
Solution

(a) We can translate from base-2 to base-4 by dividing the source bit string (of even length) into consecutive bit pairs and mapping each pair onto a base-4 digit, i.e., $0\,0 \to 0$, $0\,1 \to 1$, $1\,0 \to 2$ and $1\,1 \to 3$. For instance: $\underbrace{0\,0}\ \underbrace{1\,0}\big|_{\text{base-2}} \to 0\ 2\big|_{\text{base-4}}$. The source grammar G_s is right-linear. Nonterminals Q and R respectively correspond to the first bit 0 and 1 of a source pair, thus a base-4 digit can be generated in correspondence to the second bit of the pair, when deriving Q and R. Here are the destination and translation grammars G_d and G_τ (axiom S):

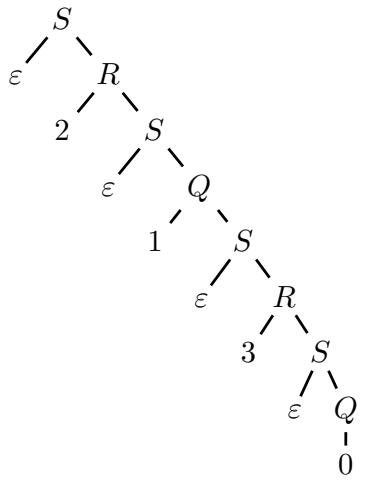
$$G_d \begin{cases} S \to Q \mid R \\ Q \to 0\,S \mid 1\,S \mid 0 \mid 1 \\ R \to 2\,S \mid 3\,S \mid 2 \mid 3 \end{cases} \qquad G_\tau \begin{cases} S \to \frac{0}{\varepsilon}\,Q \mid \frac{1}{\varepsilon}\,R \\ Q \to \frac{0}{0}\,S \mid \frac{1}{1}\,S \mid \frac{0}{0} \mid \frac{1}{1} \\ R \to \frac{0}{2}\,S \mid \frac{1}{3}\,S \mid \frac{0}{2} \mid \frac{1}{3} \end{cases}$$

(b) Here is the destination tree of the sample bit string, paired to the source tree:

source tree



destination tree



Proceed similarly for the translation tree (just overlap the two trees above).

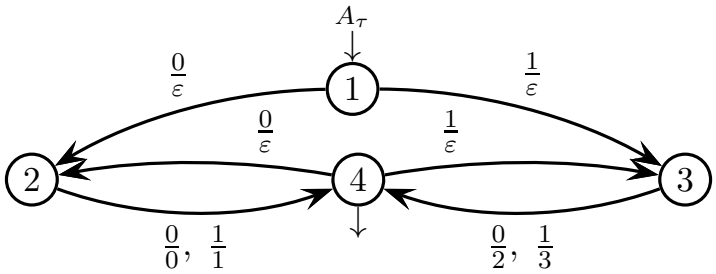
(c) Here is a rational translation expression R_τ, rather simple and intuitive:

$$R_\tau = \left(\frac{0\,0}{0} \mid \frac{0\,1}{1} \mid \frac{1\,0}{2} \mid \frac{1\,1}{3} \right)^+ \quad \text{// sequence of bit pairs and base-4 digits}$$

or equivalently, after normalizing (label $\frac{0\,0}{0}$ can be normalized as label $\frac{0}{\varepsilon}\,\frac{0}{0}$, etc):

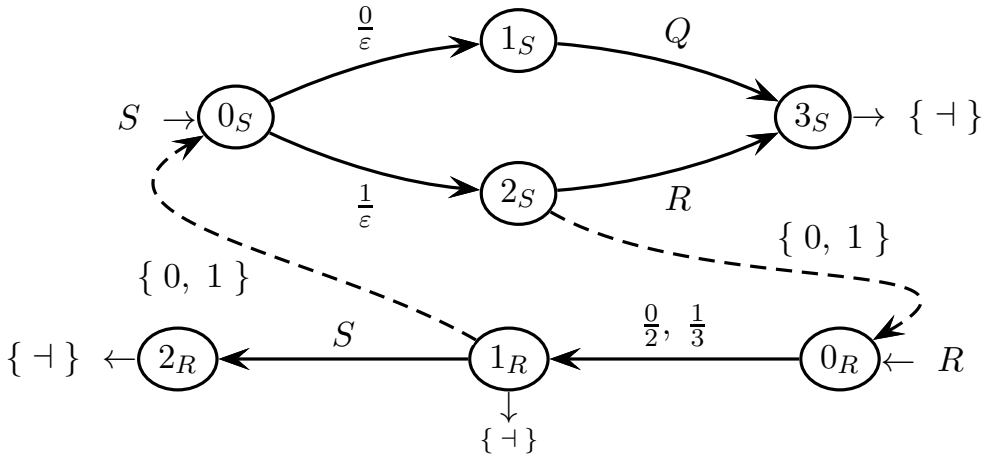
$$R_\tau^{norm} = \left(\frac{0}{\varepsilon}\,\frac{0}{0} \mid \frac{0}{\varepsilon}\,\frac{1}{1} \mid \frac{1}{\varepsilon}\,\frac{1}{2} \mid \frac{1}{\varepsilon}\,\frac{1}{3} \right)^+ = \left(\frac{0}{\varepsilon} \left(\frac{0}{0} \mid \frac{1}{1} \right) \mid \frac{1}{\varepsilon} \left(\frac{1}{2} \mid \frac{1}{3} \right) \right)^+$$

Here is a corresponding finite-state transducer A_τ (IO-automaton – four states):



Transducer A_τ is deterministic, as so is the underlying recognizer automaton. Like grammar G_τ and expression R_τ, it does not accept / translate string ε.

(d) Here are the machines M_S and M_R of a translation net for grammar G_τ, with guide sets on the call arcs and exit arrows:



The guide set on the call arc to M_Q (not shown) is identical to that on the call arc to M_R. The source grammar G_s is right-linear and of type $ELL(1)$. The requested syntactic procedures of the recursive descent analyzer are as follows:

```
procedure S
if cc = 0 then                  // state 0
    cc := next                  // go to 1
    if cc ∈ { 0, 1 } then       // state 1
        call Q                  // go to 3
        if cc = ⊣ then          // state 3
            return
        else error              // state 3
    else error                  // state 1
else if cc = 1 then             // state 0
    cc := next                  // go to 2
    if cc ∈ { 0, 1 } then       // state 2
        call R                  // go to 3
        if cc = ⊣ then          // state 3
            return
        else error              // state 3
    else error                  // state 2
else error                      // state 0
```

```
procedure R
if cc = 0 then                  // state 0
    write (2)
    cc := next                  // go to 1
    if cc ∈ { 0, 1 } then       // state 1
        call S                  // go to 2
        if cc = ⊣ then          // state 2
            return
        else error              // state 2
    else if cc = ⊣ then         // state 1
        return
    else error                  // state 1
else if cc = 1 then             // state 0
    write (3)
    cc := next                  // go to 1
    if cc ∈ { 0, 1 } then       // state 1
        call S                  // go to 2
        if cc = ⊣ then          // state 2
            return
        else error              // state 2
    else if cc = ⊣ then         // state 1
        return
    else error                  // state 1
else error                      // state 0
```

Procedure Q is structurally identical to procedure R, only the output is different.

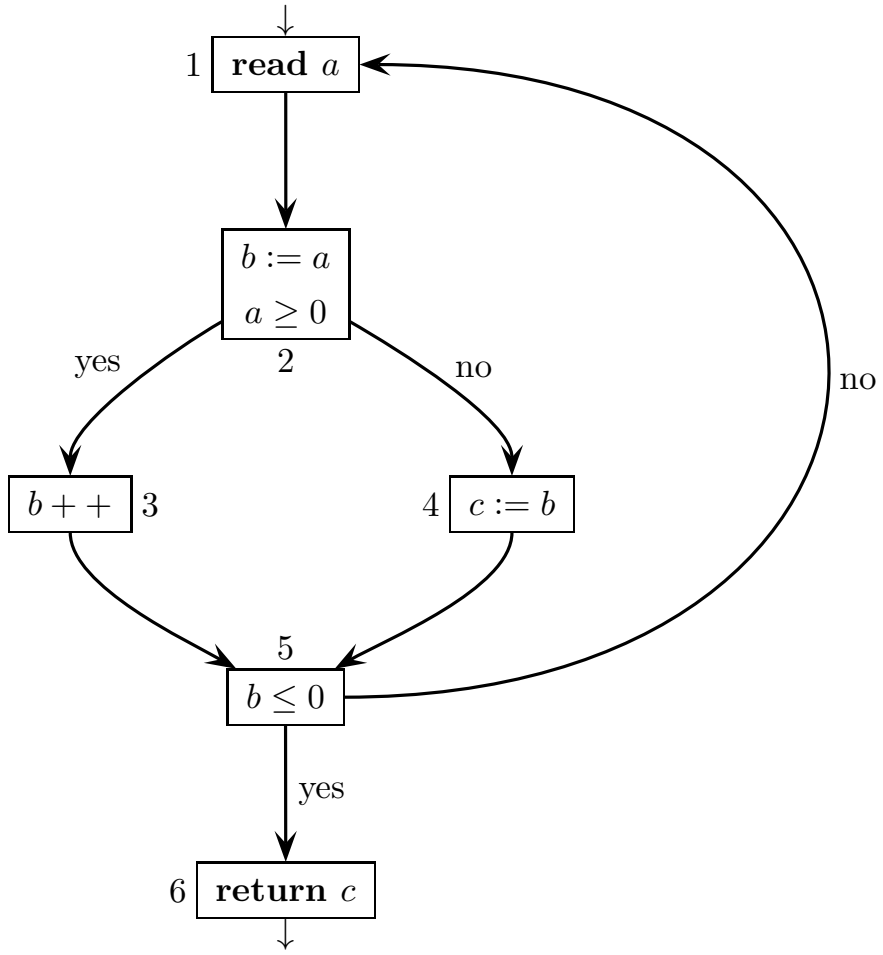
Code optimization is possible, for instance by grouping the error cases at the procedure end and by factoring identical code snippets. We obtain some significant code compaction, for both procedures and in particular for R:

```
procedure S
if cc = 0 then                  // state 0
    cc := next                  // go to 1
    if cc ∈ { 0, 1 } then       // state 1
        call Q                  // go to 3
        if cc = ⊣ then          // state 3
            return
else if cc = 1 then             // state 0
    cc := next                  // go to 2
    if cc ∈ { 0, 1 } then       // state 2
        call R                  // go to 3
        if cc = ⊣ then          // state 3
            return
error                // state 0 or 1 or 2
```

```
procedure R
if cc = 0 then                  // state 0
    write (2)
else if cc = 1 then             // state 0
    write (3)
else error                      // state 0
cc := next                      // go to 1
if cc ∈ { 0, 1 } then           // state 1
    call S                      // go to 2
if cc = ⊣ then           // state 1 or 2
    return
else error               // state 1 or 2
```

Of course, other codings are possible, more or less similar to the ones above.

2. Consider the program Control Flow Graph (*CFG*) shown below, with six nodes:



The program has three integer variables: a, b and c.

Answer the following questions (use the tables / drawings / spaces on the next pages):

(a) Disregarding the program semantics, write a regular expression R that represents the execution traces of the *CFG* over the node name alphabet $\{ 1, \ldots, 6 \}$.

(b) Informally find the live variables (on the node input) at each node and write them by each node. Can the found liveness intervals be exploited to optimize the memory allocation for the program ? Justify your answer.

(c) Find the variables defined and used at each node. Then write the flow equations (*in* and *out*) of the live variables, but only for the nodes 2, 3 and 5 (do not solve such equations). Verify that the solution found at point (b) satisfies the flow equations of nodes 2, 3 and 5.

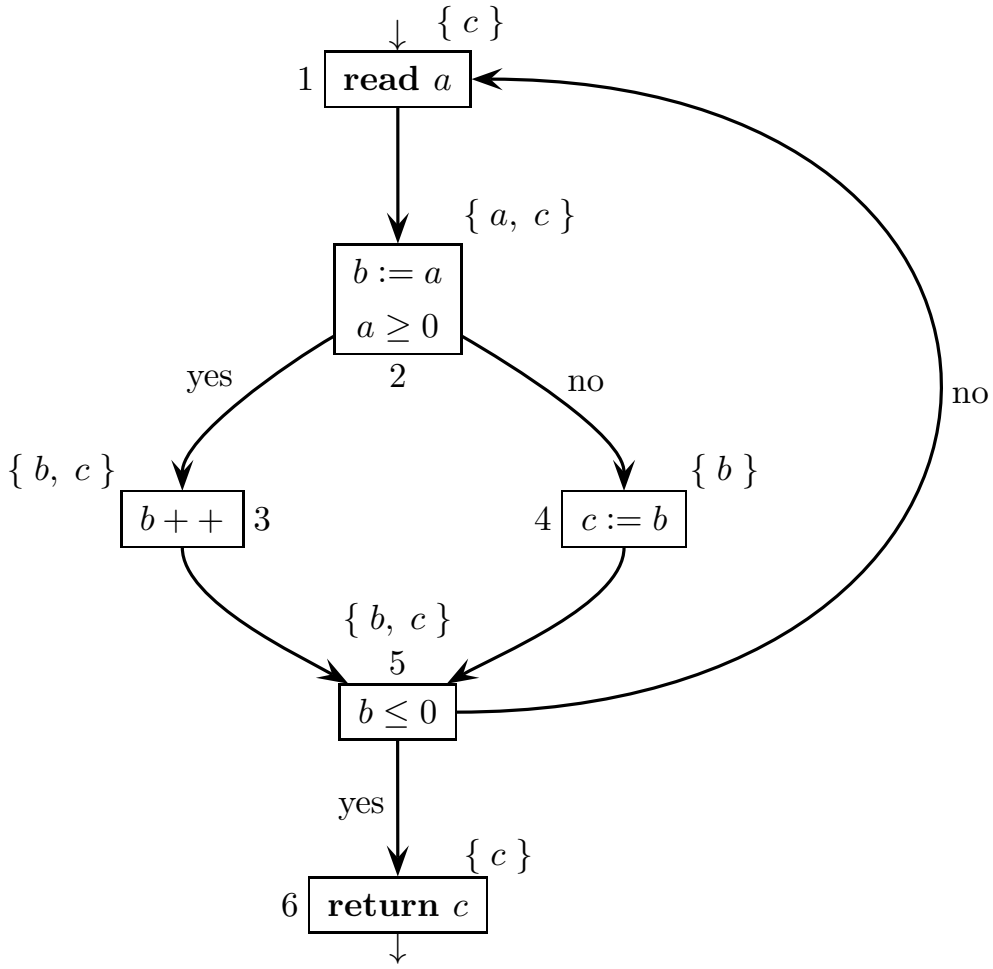
(d) (optional) List all the variable definitions. Then informally find the reaching definitions (on the node output) at each node and write them by each node. If we consider the program semantics, what changes about the reaching definitions at the input node 1 and at the output node 6 ?

Solution

(a) Here is a regular expression R for the *CFG*, disregarding program semantics:

$$R = \big(\,1\;2\;3\;5 \;\mid\; 1\;2\;4\;5\,\big)^{+}\;6 = \big(\,1\;2\;(\,3 \;\mid\; 4\,)\;5\,\big)^{+}\;6$$

(b) Here are the live variables at the *node inputs*, disregarding semantics:



Variable a is defined in the node 1, is used in the successor node 2, and is neither defined nor used anywhere else, thus it is live only at the input of node 2. All the nodes belong to a loop, thus they reach each other, except for node 6. Variable b is first defined in the node 2, is last used in the node 5, and is both used and defined in the node 3, thus it is live on all the loop paths that connect node 2 to node 5, i.e., 2 3 5 and 2 4 5. Variable c is defined in the node 4, is used in the node 6 (it may be not defined when it reaches node 6), and is neither defined nor used anywhere else. Furthermore, node 6 is reachable from any loop node without (re)defining variable c, except for node 4 (which defines c). Therefore variable c is live everywhere, except – as said – for the input of node 4.

By inspecting the liveness intervals, we see that variables a and b are never live together. Hence one memory cell or processor register is sufficient to store both.

Remark: the program code can be optimized and the variables can be reduced, by removing the assignment $b := a$ and replacing variable b with variable a.

(c) Here are the required language equations and their verification:

tables of definitions and usages at the nodes

node	*defined*
1	a
2	b
3	b
4	c
5	–
6	–

node	*used*
1	–
2	a
3	b
4	b
5	b
6	c

(partial) system of data-flow equations

node	*in equation*	*out equation*
2	$in(2) = \{\ a\ \} \cup \big(\ out(2) - \{\ b\ \}\ \big)$	$out(2) = in(3) \cup in(4)$
3	$in(3) = \{\ b\ \} \cup \big(\ out(3) - \{\ b\ \}\ \big)$	$out(3) = in(5)$
5	$in(5) = \{\ b\ \} \cup \big(\ out(5) - \emptyset\ \big)$ $= \{\ b\ \} \cup out(5)$	$out(5) = in(1) \cup in(6)$

equation verification

The verification procedure is carried out as follows: first we compute the *out*-set for a node, by using the corresponding *out*-equation and the *in*-sets informally found at point (b); then we compute the *in*-set for the same node, by using the corresponding *in*-equation and the *out*-set computed; eventually we verify that the *in*-set computed coincides with the informal one found at point (b) for the same node. This procedure is repeated for the three nodes required:

***node* 2:** $out(2) = in(3) \cup in(4) = \{\ b,\ c\ \} \cup \{\ b\ \} = \{\ b,\ c\ \}$, hence $in(2) = \{\ a\ \} \cup \big(\ \{\ b,\ c\ \} - \{\ b\ \}\ \big) = \{\ a\ \} \cup \{\ c\ \} = \{\ a,\ c\ \}$, verified

***node* 3:** $out(3) = in(5) = \{\ b,\ c\ \}$, hence $in(3) = \{\ b\ \} \cup \big(\ \{\ b,\ c\ \} - \{\ b\ \}\ \big) = \{\ b\ \} \cup \{\ c\ \} = \{\ b,\ c\ \}$, verified

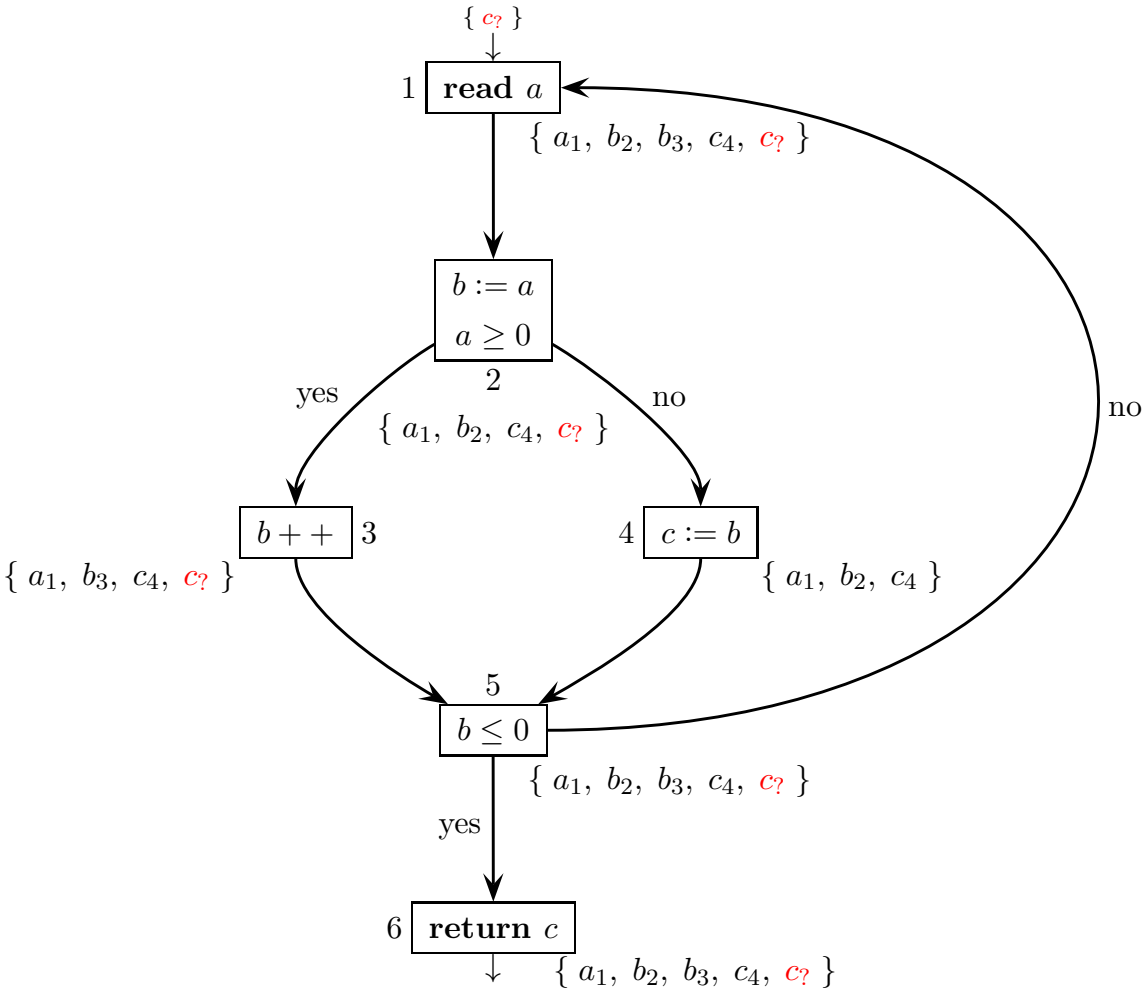
***node* 5:** $out(5) = in(1) \cup in(6) = \{\ c\ \} \cup \{\ c\ \} = \{\ c\ \}$, hence $in(5) = \{\ b\ \} \cup \big(\ \{\ c\ \} - \emptyset\ \big) = \{\ b\ \} \cup \{\ c\ \} = \{\ b,\ c\ \}$, verified

In conclusion, all the language equations for the required nodes are successfully verified with respect to the informal solution found at point (b).

(d) Here is the list of variable definitions v_i, that is, a variable v and a node i where the variable can be modified:

$a_1 \quad b_2 \quad b_3 \quad c_4$ // remember that $b{+}{+}$ is equivalent to $b := b + 1$

Here are the definitions that reach the *node outputs*, disregarding semantics:

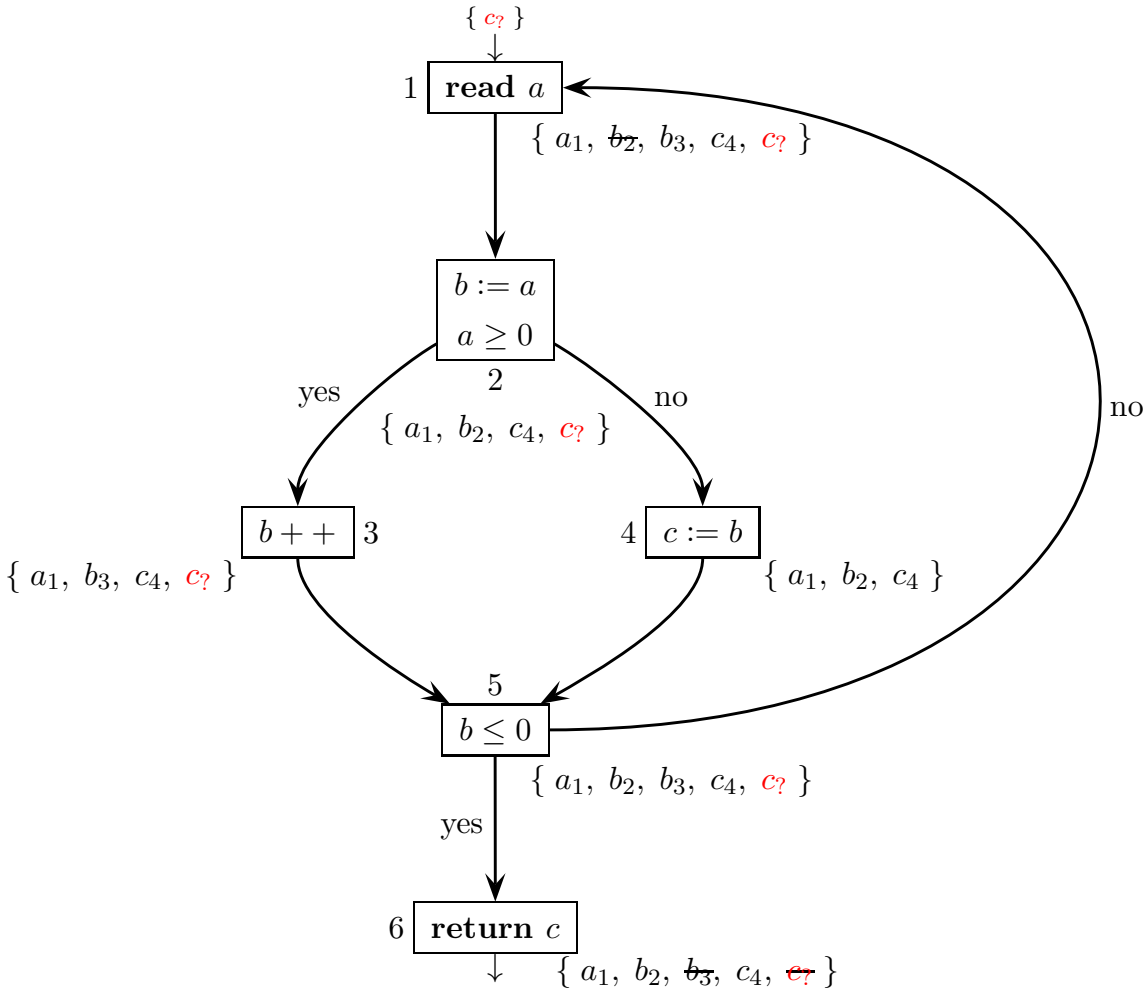


The distribution of the reaching definitions is simple to understand. Since – as said at point (b) – every node (except for 6) has a path to any other node, basically every definition reaches any node, with only a few suppression cases. In fact, just notice that the definitions b_2 and b_3 are respectively suppressed at the outputs of nodes 3 and 2, 4. Furthermore, the program might return (in the node 6) the variable c uninitialized. Thus for generality the definition of an input parameter $c_?$ might be considered as well (colored in red). Of course, the definition of $c_?$ is suppressed at the output of node 4, but it reaches the outputs of all the other nodes. There are not any other suppression cases.

Now consider program semantics. Due to the sign tests of variables a and b in the nodes 2 and 5, respectively, only the (cyclic) path 1 2 3 5 1 back to the initial node 1 and the (acyclic) path 1 2 4 5 6 to the final node 6 are possible, while both paths 1 2 4 5 1 (cyclic) and 1 2 3 5 6 (acyclic) are impossible.

Therefore definition b_2 may not reach (the output of) node 1, since it is always suppressed by definition b_3. Similarly, definition b_3 may not reach (the output of) node 6, since either it is not executed or, even if it is executed once or more times, it is ultimately suppressed by definition b_2. For the same reason, the definition of the input parameter $c_?$ may not reach the output of node 6.

For completeness, here is the *CFG* with the actual reaching definitions, considering semantics (the definitions suppressed by semantics are crossed out):



Said differently, the regular expression R of the program execution traces without semantics can be refined with semantics as follows:

$$R_{refined} = \left(1\ 2\ 3\ 5\right)^*\ 1\ 2\ 4\ 5\ 6$$

The program may loop through node 3, but after passing once through node 4, it stops looping and terminates. Clearly it holds $L\left(R_{refined}\right) \subset L\left(R\right)$, and the containment is strict. For this program, it is possible to model the effect of semantics by a sublanguage (of the regular language without semantics) that is still regular. It is a special case however, as for most programs the sublanguage of the execution traces with semantics will turn out to be non-regular.